RIO: Runtime Intelligence Orchestration

A Cryptographic Protocol for Governed AI Execution

Version: 2.0.0 **Date:** March 25, 2026 **Author / Architect:** Brian K. Rasmussen **Technical Implementation and Documentation:** Manny **Status:** Technical Whitepaper

---

1. Abstract

Runtime Intelligence Orchestration (RIO) is a fail-closed authorization and audit protocol designed to govern autonomous AI agents. As AI systems transition from passive advisors to active participants in digital environments — capable of moving funds, managing infrastructure, and accessing sensitive data — the risk of unaligned or malicious execution increases. RIO addresses this by decoupling the "intelligence" of the agent from the "authority" to execute. By enforcing a cryptographic control plane between the AI and the execution target, RIO ensures that no high-impact action can occur without explicit, verifiable human approval. The system provides a tamper-evident audit trail through a hash-chained ledger and generates cryptographic receipts for every execution. This paper details the RIO architecture, its 8-stage execution pipeline, the cryptographic security model, and its implementation as a hardened gateway for enterprise AI operations.

---

2. Introduction

The rapid advancement of Large Language Models (LLMs) has birthed a new era of autonomous agents. These agents are no longer confined to chat interfaces; they are integrated into business workflows via APIs, database connectors, and cloud infrastructure. However, this integration introduces a critical "speed asymmetry": AI can propose and attempt actions at machine speed, while human oversight remains at human speed.

Traditional security models — such as prompt engineering, system instructions, or model alignment — are advisory. They rely on the AI's "willingness" to follow rules. In a production environment, this is insufficient. A single hallucination or prompt injection can lead to irreversible consequences, such as unauthorized financial transfers or data breaches.

RIO (Runtime Intelligence Orchestration) shifts the paradigm from *advisory* to *structural* governance. It treats AI as an untrusted requester and places a hard execution gate in front of every sensitive action. By requiring a cryptographic "proof of approval" at the moment of execution, RIO ensures that the human remains the ultimate authority, without sacrificing the efficiency of AI-driven orchestration.

---

3. System Overview

RIO is built on the principle that **AI proposes, but the system executes.** It operates across two distinct planes:

The Control Plane: This plane governs the flow of intent. It captures the AI's request, evaluates it against organizational policies, determines the required risk level, and manages the human approval workflow. It is responsible for generating the cryptographic "Execution Token" that unlocks the gate.

The Audit Plane: This plane provides the "memory" of the system. It records every intent, approval, denial, and execution event in a tamper-evident, hash-chained ledger. It generates "Cryptographic Receipts" that allow any party to independently verify that an action was authorized and executed correctly.

The system is **fail-closed** by design. If the control plane is unavailable, if a signature is invalid, or if the ledger cannot be written, the execution gate remains locked. This ensures that no action is ever taken in an unrecorded or unauthorized state.

---

4. Architecture

The RIO architecture is divided into four trust zones, each with clearly defined boundaries and responsibilities:

1. **External Requesters (Untrusted):** This zone includes AI agents, automated scripts, and frontend clients. They can submit “Intents” but have no direct access to execution targets or cryptographic keys.

2. **RIO Control Plane (Trusted):** The core of the system. It contains the Intent Service, Policy Engine, and Signature Service. It manages the transition from an “Intent” to an “Authorized Action.”

3. **Execution Targets (Protected):** These are the real-world systems (e.g., Gmail API, Stripe, AWS) that perform the actions. They are shielded by the Execution Gate, which only accepts requests accompanied by a valid RIO Execution Token.

4. **Audit & Attestation (Immutable):** This zone houses the Ledger and Receipt Service. It provides the source of truth for all system activity and is designed to be tamper-evident.

Trust Boundaries:

The boundary between Requesters and the Control Plane is secured by standard authentication and the “Intake” protocol. The boundary between the Control Plane and Execution Targets is the “Hard Gate,” secured by ECDSA (secp256k1) signatures. The boundary between the Control Plane and Audit Plane is an append-only, hash-chained connection.

---

5. The Governed Execution Pipeline

RIO enforces governance through a rigorous 8-stage pipeline. Each stage produces a specific data structure that is passed to the next, ensuring a continuous chain of custody.

1. **Intake:** The AI agent submits a raw intent (e.g., “Send $500 to Alice”).
2. **Classification:** The system identifies the action type and extracts parameters.
3. **Structured Intent:** The intent is converted into a machine-readable format with a unique `intent_id`.
4. **Policy & Risk Evaluation:** The Policy Engine checks the intent against active rules. A risk score is calculated based on the action, amount, and target.

5. **Authorization:** If the risk exceeds the threshold, a human approver is notified. Upon approval, the Signature Service generates an ECDSA signature.
6. **Execution Gate:** The requester submits the action and the Execution Token (containing the signature) to the gate. The gate verifies the signature, timestamp, and nonce.
7. **Receipt Generation:** Upon successful execution, an HMAC-signed receipt is generated, capturing the result and anchoring it to the current ledger state.
8. **Ledger Entry:** The event is recorded in the hash-chained ledger, linking it to the previous entry's hash.

Pipeline Flow Diagram:

```
[ AI Agent ] --(Intent)--> [ Intake ]
                                |
                                ▼
                        [ Classification ]
                                |
                                ▼
                    [ Policy & Risk ] --(High Risk)--> [ Human Approval ]
                                |                                |
                                ▼                                ▼
                    [ Execution Gate ] <---(Execution Token)--┘
                                |
                  ┌─────────────┴─────────────┐
                  ▼                           ▼
            [ If Denied ]             [ If Approved ]
            (Block & Log)            (Execute Action)
                                           |
                                           ▼
                                  [ Receipt & Ledger ]
```

---

6. System Invariants

The RIO protocol is governed by ten core invariants that must be maintained at all times to ensure system integrity:

1. **No Execution Without Authorization:** No action can be performed by an execution target unless a valid, unconsumed Execution Token is presented.

2. **No Authorization Without Policy Check:** An Execution Token can only be generated after the Policy Engine has evaluated the intent.
3. **Fail-Closed Enforcement:** Any failure in a dependency (database, signature service, ledger) must result in a blocked action.
4. **Single-Use Approvals:** Every Execution Token and its associated signature are single-use. Replay attacks are blocked at the structural level.
5. **Cryptographic Binding:** The signature must be bound to the exact payload (action + parameters) presented for approval.
6. **Timestamp Freshness:** Execution Tokens have a maximum lifespan (default 300s) to prevent the use of stale authorizations.
7. **Every Action Produces a Receipt:** Every execution attempt, whether successful or blocked, must generate a cryptographic receipt.
8. **Tamper-Evident Audit Trail:** All receipts must be recorded in a hash-chained ledger where each entry anchors to the previous.
9. **Identity Attribution:** Every action must be attributed to both the requesting agent and the authorizing human.
10. **Immutable History:** Ledger entries cannot be modified or deleted; the chain must be verifiable from the genesis block.

---

7. Cryptographic Audit Model

RIO uses a multi-layered cryptographic model to ensure that the audit trail is both authentic and tamper-evident.

Receipt Structure: A RIO receipt is a JSON object containing the full context of the execution. It is signed using HMAC-SHA256 with a key known only to the RIO Control Plane.

```json
{
  "intent_id": "INT-1042",
  "action": "send_email",
  "timestamp": "2026-03-25T12:00:00Z",
  "approver": "brian_rasmussen",
  "agent": "finance_bot_v1",
  "executed_by": "RIO Control Plane",
  "policy_result": "APPROVED",
  "parameters_hash": "sha256(...)",
  "result_hash": "sha256(...)",
  "ledger_hash": "prev_entry_hash",
  "signature": "hmac_signature"
}
```

Ledger Structure: The ledger is a hash chain where each entry E_n contains a hash H_n calculated as: `H_n = SHA256(E_n.data + H_(n-1))`. This structure ensures that any modification to entry E_i will invalidate all subsequent hashes H_j (where j >= i), making tampering immediately detectable.

Verification: The system provides a `verify_ledger_integrity()` function that traverses the chain, recomputing hashes and checking continuity. Receipts can be independently verified by recomputing the HMAC signature using the system's receipt key.

---

8. Threat Model

RIO is designed to mitigate critical threats in autonomous AI environments. The following table summarizes the primary threat categories and their structural mitigations:

Threat Category	Attack Vector	Mitigation
Unauthorized Execution	Direct call to executor API	Service boundary + service-to-service auth
Ledger Tampering	Database modification	Hash-chained ledger entries
Token Reuse	Replay of valid token	Single-use nonce/signature registry
Privilege Escalation	Forged approval record	Independent ECDSA signature verification
Kill Switch Bypass	Disabling the gate	Fail-closed design (no gate = no execution)
Missing Audit Trail	Execution without logging	Ledger write as a prerequisite for execution

---

9. Governance Model

The RIO Governance Model defines how rules are created, evaluated, and updated.

Policy Engine: Policies are defined as a set of rules that map actions and parameters to risk levels. The engine evaluates intents in real-time, returning a verdict of `ALLOW`, `BLOCK`, or `REQUIRE_APPROVAL`.

Risk Engine: Risk is calculated using a 4-component scoring model:

1. **Base Risk:** The inherent risk of the action type (e.g., `send_email` vs `move_funds`).
2. **Role Modifier:** Adjusts risk based on the agent's assigned role.
3. **Amount Modifier:** Scales risk based on financial or data volume.
4. **Target Modifier:** Adjusts risk based on the sensitivity of the target system.

Policy Lifecycle: Policies follow a strict versioning lifecycle: `PROPOSED` → `APPROVED` → `ACTIVATED` → `INACTIVE` (or `ROLLED_BACK`). Only one policy version can be `ACTIVATED` at any time, ensuring deterministic evaluation.

---

10. Identity and Access Control

RIO implements a robust Identity and Access Management (IAM) system tailored for AI-human collaboration.

Role Hierarchy: The system supports a 5-role hierarchy for human users:

1. **Intern:** View-only access to logs.
2. **Employee:** Can propose intents and approve low-risk actions.
3. **Manager:** Can approve medium-risk actions.
4. **Director:** Can approve high-risk actions and modify policies.
5. **Admin:** Full system control, including key management.

Authorization Tokens: Execution Tokens are cryptographically bound to the user's identity and the specific intent. They are time-bound (default 300 seconds), single-use (consumed upon first presentation to the gate), and nonce-protected (prevents replay attacks).

---

11. Learning and Simulation

RIO includes a "Governed Corpus" that records all system interactions, providing a rich dataset for learning and policy refinement.

Replay Engine: The system can "replay" historical intents through the pipeline in three modes:

1. **Exact Replay:** Verifies that the system produces the same result for the same input.
2. **Modified Policy:** Simulates how a new policy would have handled past intents.
3. **Modified Role:** Simulates how different user roles would have impacted the outcome.

Policy Improvement Loop:

1. **Record:** Capture intents and outcomes in the governed corpus.
2. **Analyze:** Identify patterns of friction or risk.

3. **Simulate:** Test new rules against the corpus.
4. **Deploy:** Activate refined policies with confidence.

---

12. Multi-Agent Governance

In complex environments, AI agents often interact with one another. RIO governs these interactions using the Governed Agent Pattern.

When Agent A requests an action from Agent B, the request is treated as an "Intent" by RIO. If the action is high-risk, RIO intercepts the flow and requires human approval before Agent B can execute. This ensures that delegation between agents does not bypass the human authority gate.

Agent-to-RIO Workflow: `Agent A → Agent B → RIO Client → Pipeline → Approval → Execution → Result → Agent A`.

---

13. Enterprise Use Cases

RIO is designed for high-stakes enterprise environments. Below are five key scenarios:

1. **Invoice Payment Approval:** A finance agent identifies an outstanding invoice. RIO intercepts the payment request, requiring a Manager's biometric approval for any amount over $1,000.
2. **GDPR Data Deletion:** An agent tasked with data privacy receives a deletion request. RIO ensures the deletion is logged and verified against the correct user ID before execution.
3. **Production Deployment:** A DevOps agent proposes a code deployment. RIO requires a Director-level signature, ensuring that no code reaches production without a human "go" decision.
4. **Access Provisioning:** An HR agent requests system access for a new hire. RIO validates the request against the employee's role and requires Admin approval for privileged access.

5. **Agent-to-Agent Delegation:** A personal assistant agent asks a travel agent to book a flight. RIO gates the final payment, ensuring the user approves the cost and itinerary.

---

14. Implementation Status

The RIO system is a fully functional, hardened implementation. The current repository includes:

- **Core Pipeline:** An 8-stage Python implementation (`pipeline.py`) with full data structure support.
- **Hardened Gateway:** A FastAPI-based gateway with ECDSA enforcement and nonce registry.
- **Cryptographic Services:** ECDSA (secp256k1) signing and HMAC-SHA256 receipt generation.
- **Tamper-Evident Ledger:** A file-based hash-chained ledger with integrity verification tools.
- **Test Suite:** 47 passing tests across 12 suites, covering all critical security vectors (V-001 to V-010).
- **Admin Dashboard:** A frontend UI for monitoring the audit log and managing policies.
- **Policy & Risk Admin:** Full versioning, approval, activation, and rollback for policies and risk models.
- **Governed Agent Example:** A complete example showing how AI agents integrate with RIO.
- **Simulation Engine:** Replay engine with exact, modified-policy, and modified-role modes.

---

15. Limitations and Future Work

While RIO provides a robust foundation for AI governance, several areas are identified for future development:

- **Distributed Ledger:** Moving from a single-node file-based ledger to a distributed, decentralized ledger for enhanced resilience.
- **Real Adapter Integrations:** Expanding the library of execution adapters beyond simulated tools to include direct integrations with major enterprise APIs.
- **Multi-Tenant Support:** Enhancing the architecture to support multiple organizations within a single RIO instance.
- **Hardware Security Modules (HSM):** Integrating with HSMs for the secure storage and management of RIO's master signing keys.

---

16. Conclusion

RIO (Runtime Intelligence Orchestration) provides the missing link in AI safety: a structural enforcement layer that operates at the speed of machine intelligence while maintaining the absolute authority of human decision-makers. By decoupling intent from execution and anchoring every action in a cryptographic audit trail, RIO enables organizations to deploy autonomous agents with confidence. The key insight of RIO is that governance does not have to be a bottleneck; it can be a verifiable, tamper-evident, and automated part of the execution itself.

---

17. References

- RIO Repository: github.com/bkr1297-RIO/rio-protocol
- RIO Protocol Specification: `/spec/governed_execution_protocol.md`
- NIST AI Risk Management Framework (AI RMF 1.0)
- ISO/IEC 42001:2023 — Information technology — Artificial intelligence — Management system
- ECDSA (secp256k1) Standards (SEC 2)